CONCEPTUAL USE-CASE

Process Intelligence for *Jira* with Petri Nets

A formal-verification layer that sits on top of your Jira board, detects bottlenecks, deadlocks and liveness violations in real-time, and pushes actionable recommendations back via Claude Code and MCP.

01 — Problem

Why Jira boards lie to you

Jira shows you *what* is happening — a kanban column, a sprint report, a velocity chart. It does not tell you *why* tickets pile up in review, why the same story gets reopened three times, or whether the current workflow can ever deliver all committed stories before the sprint ends.

Petri nets are a mathematical model for concurrent processes invented by Carl Adam Petri in 1962. They model states as **places**, actions as **transitions**, and work items as **tokens**. With that model in hand you can *formally prove* properties of your development process — the same way a compiler proves type safety.

DEADLOCK

A state from which no transition can ever fire — the process grinds to a halt with tickets stuck and no defined next step.

LIVENESS VIOLATION

Some transition never fires from any reachable state — a workflow path that exists on paper but is never actually used.

UNBOUNDEDNESS

A place accumulates tokens without limit — equivalent to a WIP explosion where reviewers or QA become an infinite queue.

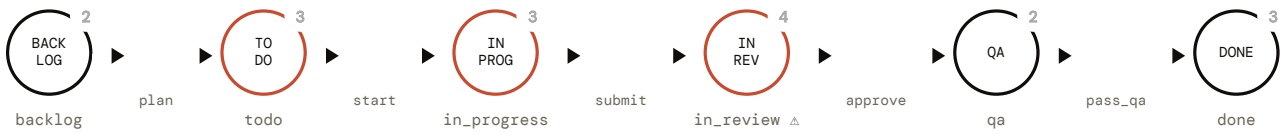
REVERSIBILITY

The ability to return to the initial marking — relevant when a "Done" ticket needs to be reopened due to a regression.

02 — Model

The Jira workflow as a Petri net

Each Jira status becomes a **place**. Each allowed status transition becomes a **transition bar**. The number of tickets currently in that status becomes the **token count**.



Red badge = token count (live ticket count from Jira). Orange border = WIP accumulation alert. Side-transitions (rework, fail_qa, block/unblock, reopen) omitted for clarity.

MAPPING RULES

`Jira status`	→ **Petri net Place** Backlog · To Do · In Progress · In Review · QA · Done · Blocked
`Ticket count`	→ **Token count in that Place** Fetched via Jira MCP `search_issues` and grouped by status.
`Status change`	→ **Transition fire** plan · start · submit · rework · approve · fail_qa · pass_qa · block · unblock · reopen
`Blocked-by link`	→ **Token in "blocked" place + dependency arc** A blocked ticket holds a token in the `blocked` place until its dependency resolves.

---

03 — Architecture

Claude Code + MCP + Petri net engine

Three components integrate through a simple daily automation loop triggered by CI/CD or a cron job inside Claude Code.

JIRA MCP SERVER

Read-write access to the project board via the official Jira MCP adapter. Provides `search_issues`, `add_comment`, `create_issue`, and `transition_issue` tools.

CLAUDE CODE (ORCHESTRATOR)

Runs the Python pipeline, interprets the analysis output in natural language, decides which Jira actions to take, and posts human-readable

PETRI NET ENGINE

`petri_net.py` — runtime. `petri_net_analysis.py` — formal verifier (BFS reachability, SCC liveness, siphon/trap enumeration).

	recommendations back to the board.	`jira_petri_workflow.py` — bridge.

DATA FLOW

1	**MCP pull**	Claude Code calls `jira.search_issues(jql="sprint = active")` via MCP. Returns JSON list of issues with status, assignee, created date, blocked-by links.
2	**Net build**	`build_net_from_jira(issues)` groups tickets by status, sets token counts on each place, and wires the full transition graph.
3	**Analysis**	`analyse(net)` runs boundedness, safeness, deadlock-freedom, quasi-liveness, full L1-liveness (Kosaraju SCC), reversibility, and siphon/trap checks.
4	**Insights**	`derive_insights()` translates formal results into human-readable findings ranked CRITICAL / WARNING / INFO with affected ticket keys.
5	**MCP push**	Claude Code posts the report as a comment on the Sprint Epic. CRITICAL findings trigger automatic `create_issue` calls that assign follow-up tasks to the scrum master.
6	**Explanation**	Claude replies in plain English — "Your review queue is a siphon, meaning once tickets enter it and reviewers are occupied, the net can deadlock if no rework arc exists" — so the team understands the *why*, not just the *what*.

---

04 — Insights

What the analysis surfaces on a real sprint

Running against a representative 19-ticket sprint (3 in Backlog, 3 To Do, 3 In Progress, 4 In Review, 2 QA, 3 Done, 2 Blocked) the engine produced:

🔴 CRITICAL	**2 blocked tickets (DEV-151, DEV-152)** Formally: tokens are in the "blocked" place with no outgoing enabled transition until a dependency place receives a token first. Action: unblocking session with named owner + deadline.
🟡 WARNING	**WIP accumulation in "in_review" (4 tickets)** Formally: the "in_review" place is unbounded in the partial state space — tokens arrive faster than the "approve" transition fires. Introduce a WIP limit of 3.
🟡 WARNING	**Review queue 1.3× larger than In Progress**

	Formally: the input rate to "in_review" dominates the output rate of "approve". Reviewers will become a structural bottleneck within 1–2 more transitions.
🟡 WARNING	**Sprint throughput 18% (3/17 done)** Formally: the proportion of tokens in the "done" place is critically low. The remaining marking cannot drain to "done" without a flow-rate change. Descope now.
🟢 INFO	**Process irreversibility detected** Formally: M_0 is not reachable from some markings (expected for Done tickets). Confirm that Done is the only intended terminal state; add a Reopen transition if needed.

---

05 — Code

Core implementation sketch

1 — BUILD THE NET FROM LIVE JIRA DATA

```
# jira_petri_workflow.py  (abridged)
def build_net_from_jira(issues):
    counts = {p: 0 for p in STATUS_TO_PLACE.values()}
    for issue in issues:
        counts[STATUS_TO_PLACE[issue.status]] += 1

    net = PetriNet("Jira Sprint Workflow")
    in_progress = net.add_place("in_progress", tokens=counts["in_progress"])
    in_review   = net.add_place("in_review",   tokens=counts["in_review"])
    # … all places …

    submit  = net.add_transition("submit")   # In Progress → In Review
    rework  = net.add_transition("rework")   # In Review → In Progress
    approve = net.add_transition("approve")  # In Review → QA

    net.add_arc(in_progress, submit);  net.add_arc(submit,  in_review)
    net.add_arc(in_review,   rework);  net.add_arc(rework,  in_progress)
    net.add_arc(in_review,   approve); net.add_arc(approve, qa)
    return net
```

2 — RUN FORMAL VERIFICATION

```
results = analyse(net, max_states=50_000)
# Returns AnalysisResult objects for:
# Boundedness · Safeness · Deadlock-freedom · Quasi-liveness
# L1-Liveness (Kosaraju SCC) · Reversibility · Siphons & Traps
```

3 — POST BACK VIA JIRA MCP

```python
# In a real Claude Code session:
result = await mcp.call_tool(
    "jira", "add_comment",
    {"issueKey": epic_key, "body": export_jira_comment(insights)}
)

# For CRITICAL findings, open a follow-up task:
if any(i.severity == "CRITICAL" for i in insights):
    await mcp.call_tool("jira", "create_issue", {
        "project": "DEV",
        "summary": "[PROCESS] Petri net detected critical issue",
        "description": report_markdown,
        "assignee": scrum_master
    })
```

4 — CLAUDE CODE ORCHESTRATION PROMPT

```
"""
You are a software process analyst in the team's CI/CD pipeline.
Your tools:
  petri_net.py · petri_net_analysis.py · jira_petri_workflow.py · Jira MCP

Daily task:
  1. Fetch live sprint data via MCP
  2. Build Petri net from current board state
  3. Run formal verification (analyse)
  4. Derive insights and rank by severity
  5. Post report to Jira Epic via MCP
  6. Create tasks for CRITICAL findings

Explain findings in plain English — the team needs to understand
WHY the math says the process has a problem, not just WHAT failed.
"""
```

---

06 — Properties

What each formal check means for your team

DEADLOCK-FREEDOM

Every reachable board state has at least one enabled transition. If false: there exists a configuration of tickets where the team has nothing valid to do next — a systemic process gap, not a people problem.

LIVENESS (L1)

From any board state, every workflow transition can eventually fire again. If false: some step (e.g. "fail_qa") gets stuck and can never happen — meaning QA can never reject a ticket from that state forward.

BOUNDEDNESS

QUASI-LIVENESS

Every column has a finite maximum ticket count across all reachable states. Unbounded columns are WIP bombs: mathematically guaranteed to grow without limit unless the upstream flow rate is capped.

Every transition fires at least once from the initial marking. A cheaper check than full liveness — confirms that no workflow step is completely unreachable given today's board snapshot.

REVERSIBILITY

The initial marking (start of sprint) is reachable from every reachable state. Expected to fail for Done tickets — but unexpected failures reveal hidden dead-ends where a ticket can never progress nor be reset.

SIPHONS & TRAPS

Siphons (once empty, stay empty) map to stages that, once cleared, never refill. Traps (once marked, stay marked) map to stages that permanently accumulate work — both are structural bottleneck predictors.

07 — Extensions

Where to take this next

Timed Petri nets — attach a firing time to each transition (e.g. average review duration from Jira cycle-time data) to predict sprint completion probability, not just structural feasibility.

Coloured Petri nets — give each token an identity (the actual issue key, priority, story points). The analysis then answers: "Will DEV-181 specifically be able to reach Done given the current board?" rather than reasoning about anonymous tokens.

Historical replay — load the Jira changelog to reconstruct the marking at every point in past sprints and run the analyser retroactively. Identify *which sprint events* historically triggered deadlock-prone states.

Live dashboard — expose the net state via a WebSocket so the team's standup screen shows the Petri net diagram updating in real-time as tickets move, with colour-coded bottleneck alerts.

WIP limit enforcement — use the MCP `transition_issue` tool to *block* Jira transitions that would push a column over its k-bounded limit, turning the formal model into a live process guard.